Phase7 Deployment

Phase 7 — Deployment

7.1 UML deployment diagram

The application is a locally deployed C++17 console application. SQLite is embedded in-process; there is no separate database server in this release. Submission files are stored in the application's local storage directory. Back up the database and submission directory together.

```
@startuml
node "School workstation / server" {
  artifact "ModeratedLMS.exe" as app
  artifact "OpenSSL Crypto library" as ssl
  database "lms.db (SQLite)" as db
  folder "storage/submissions/" as files
  app --> ssl : PBKDF2 / secure random salt
  app --> db : SQLite prepared statements
  app --> files : submission file copy
}
actor Student
actor Lecturer
actor "Head of Department" as HoD
actor Administrator
Student --> app : login / submit / view marks
Lecturer --> app : assignments / marking / extensions
HoD --> app : moderate / release
Administrator --> app : accounts / courses / enrolment
@enduml
```

7.2 Installation guide for a school technician

Requirements

- Windows 10/11 64-bit or a supported Linux distribution.
- C++17 build toolchain for building from source; prebuilt release can run without CMake if runtime dependencies are present.
- CMake 3.20+, Ninja, GCC/MinGW-w64 or MSVC, and OpenSSL Crypto development/runtime files for source builds.
- A writable application data directory with restricted access and enough space for submissions and backups.

Build from source (Windows/MSYS2 UCRT64)

1. Install MSYS2 and update it.
2. Install GCC, CMake, Ninja and OpenSSL packages as described in `README.md`.
3. Open the project root in VS Code or MSYS2 terminal.
4. Configure: `cmake -S . -B build -G Ninja -DCMAKE_PREFIX_PATH=C:/msys64/ucrt64`.
5. Build: `cmake --build build`.
6. Test: `ctest --test-dir build --output-on-failure`.

7. Run: `build\ModeratedLMS.exe` from PowerShell (or `./build/ModeratedLMS.exe` from MSYS2).

First deployment

8. Copy the release directory to a controlled location, not a public shared folder.
9. Restrict write/read permissions to authorised school staff and the application service account.
10. Launch the program and create the initial administrator account when prompted.
11. Administrator creates HoD, lecturer and student accounts; creates departments/courses; assigns lecturers and enrols students.
12. Run a smoke test with test accounts before importing or entering real learner data.
13. Schedule encrypted backups of the SQLite database and `storage/submissions/`; test restoration regularly.
14. Record the installed release tag, database backup date and responsible technician.

Upgrade and rollback

- Back up the database and submission storage before upgrade.
- Stop the application, install the new binary/source release, and retain the prior release directory.
- Start the new version and run the smoke tests.
- If validation fails, stop the new version and restore the previous binary and matching backup. Do not restore only the database or only submission files.

7.3 User manual — Student

15. Log in using the username and password supplied by the school.
16. Choose **View courses** to see enrolled courses.
17. Choose **View assignments** to see assignment titles, deadlines and maximum marks. `(EXTENDED)` identifies a changed effective deadline.
18. Choose **Submit/resubmit assignment**, enter the assignment ID and the full path to the file.
19. Choose **View released marks** to see marks after HoD release.
20. Log out. Never share credentials or attempt to access another student's account/files.

7.4 User manual — Lecturer

21. Log in and choose **View my courses**.
22. Choose **Create assignment** and enter the course ID, title, instructions, deadline and maximum mark.
23. Use **Grant individual extension** for one enrolled student, or **Grant whole-class extension** for all students enrolled in the assignment's course. Supply a valid date/time and a reason.
24. Choose **View unmarked submissions** to identify submissions.
25. Choose **Mark submission**, enter the submission ID, raw mark, feedback and reason.
26. Marks are sent for HoD moderation; lecturers cannot release marks.
27. Log out after work is complete.

7.5 User manual — Head of Department

28. Log in using the HoD account.

29. Choose **View marks for moderation**.
30. Review the submission, mark and feedback through the school's approved process.
31. Choose **Approve mark** or **Return mark** and record a meaningful reason.
32. Choose **Release approved mark** only after approval and departmental checks.
33. Do not share assessment records outside authorised channels.

7.6 User manual — Administrator

34. Log in using the administrator account.
35. Create user accounts and assign the correct role.
36. Create departments and assign the HoD.
37. Create courses and assign an active lecturer.
38. Enrol students in the correct course.
39. Deactivate accounts when access is no longer authorised; reactivate only after approval.
40. Maintain backups and ensure database/submission folders have restricted permissions.
41. Do not use the administrator role to alter marks or bypass moderation.

7.7 Release notes

v1.0 — Phase 4 baseline

- C++17 console LMS, four roles, SQLite persistence, repository/data-access layer, OpenSSL PBKDF2 password hashes, submission/resubmission, late penalty, individual extension, moderation, release and mark audit.

v1.1 — CR1 maintenance release

- Lecturer can grant a whole-class extension for an assignment.
- Class extension is stored separately from individual extensions with lecturer, timestamp, reason and new deadline.
- Each enrolled student's effective deadline includes the latest applicable class or individual extension.
- Student assignment list displays the effective deadline and `(EXTENDED)` indicator.
- Added CR1 positive and negative automated tests.

7.8 Release tagging commands

```
git tag -a v1.0 -m "Moderated LMS v1.0"
git push origin v1.0
# after CR1 changes are merged and tested:
git tag -a v1.1 -m "Moderated LMS v1.1 - class-wide extensions"
git push origin v1.1
```

Tags are commands for the repository maintainer to execute; this archive does not claim that a remote repository tag has already been pushed.

Rendered diagram: docs/deployment_diagram.png (the editable PlantUML source is shown above).

UML Deployment Diagram

